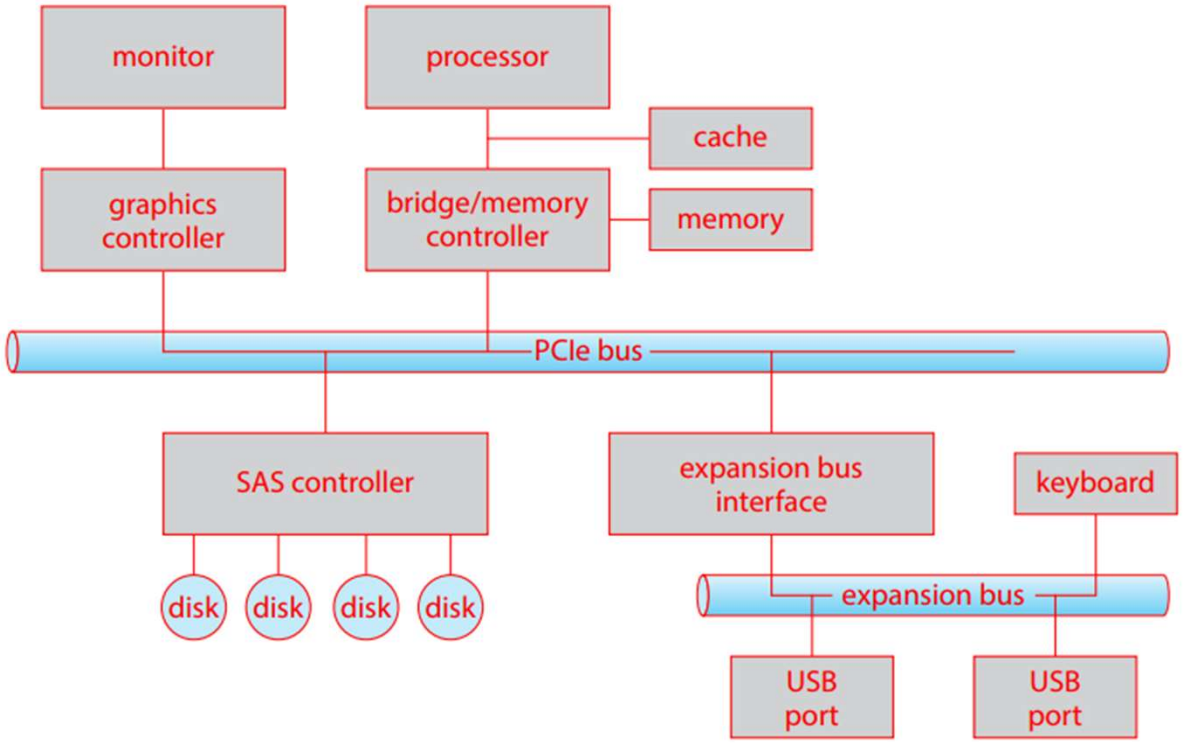
I/O and Display Interfaces

Overview

- **The Role of the OS:** The OS's main job is to **manage and control I/O operations** and devices. It acts as a critical layer between applications and the complex world of hardware.
- **Abstraction:** The OS uses **abstraction, encapsulation, and software layering** to hide the complex and varied details of different I/O devices from the rest of the system.
- **Device Drivers:** The key to this abstraction is the **device driver**. These are kernel modules that are custom-tailored for a specific device but present a **uniform interface** to the I/O subsystem. This modular design allows new devices to be added without rewriting the entire OS.

Typical Bus Structure



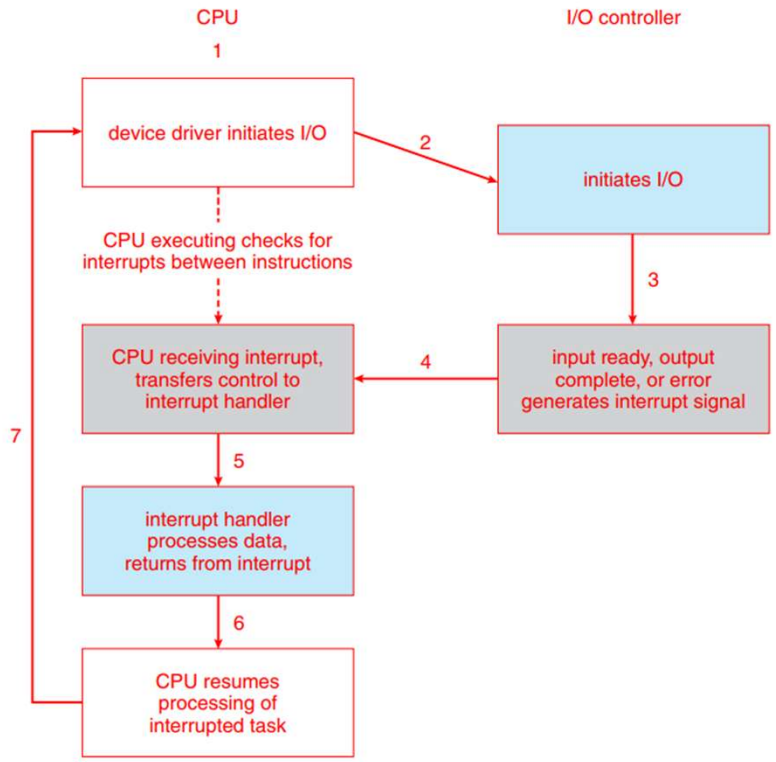
I/O Hardware

- **Ports and Buses:** Devices connect to a computer through a **port** (a connection point) or a **bus** (a shared communication pathway)
  - A common example is the PCIe bus
  - Devices can also be connected in a **daisy chain**
- **Controllers:** A **controller** is an electronic circuit that operates a port, a bus, or a device
  - The **host controller** is on the computer's side, while the **device controller** is built into the device itself
- **Memory-Mapped I/O:** Common method for communication between the CPU and a controller
  - The controller's registers are mapped to specific memory addresses
  - The CPU then issues commands and transfers data by reading and writing to these memory locations

I/O Hardware

- **Polling:** The host CPU repeatedly checks a device's status register in a loop until a "busy" bit is clear, indicating the device is ready
  - This is efficient for very fast devices but wasteful if the wait is long.
- **Interrupts:** This is a more efficient method for slower devices
  - A device controller **raises an interrupt** by sending a signal to the CPU when it needs service (e.g., an I/O operation is complete).
  - The CPU saves its current state and jumps to an **interrupt handler routine** via the **interrupt vector**.
  - The handler services the device, and the CPU resumes its original task.
  - **Interrupt priority levels** ensure that high-priority interrupts are handled first.
- **Direct Memory Access (DMA):** For large data transfers (like from a disk drive), the CPU offloads the work to a **DMA controller**
  - The DMA controller handles the data transfer directly between the device and memory without CPU intervention. This dramatically reduces CPU overhead.

Interrupt Driven I/O Cycle



Application I/O Interface

Abstraction and Encapsulation: The OS abstracts away hardware differences by grouping devices into a few conventional types, each accessed through a standardized interface

Access Conventions: The OS provides different access conventions based on device type:

- **Block Devices:** Accessed randomly in fixed-size blocks (e.g., disk drives)
- **Character Devices:** Accessed as a stream of bytes (e.g., keyboards, printers)
- **Network Devices:** Accessed through interfaces like sockets, which handle network-specific details
- **Memory-Mapped Files:** A file is treated as an array of bytes in main memory
- **Clocks and Timers:** The OS provides services to get the current time, get the elapsed time, and set timers to trigger operations at a specific time

Blocking I/O

- **How it works:** In **blocking I/O**, the application process issues a system call to perform an I/O operation (e.g., read() or write())
  - The process is then **put into a waiting state** and is blocked until the I/O operation is completed
  - The process cannot continue to execute any other code until the data transfer is finished
- This approach is simple to implement but can be inefficient, especially in a single-threaded environment
  - If a program needs to perform multiple I/O operations, it has to wait for each one to finish sequentially, which can lead to poor performance and a "frozen" user interface
- **Example:** When a program reads a file from a slow disk, the process is blocked until all the requested data has been copied into memory
  - The program cannot do anything else during this time

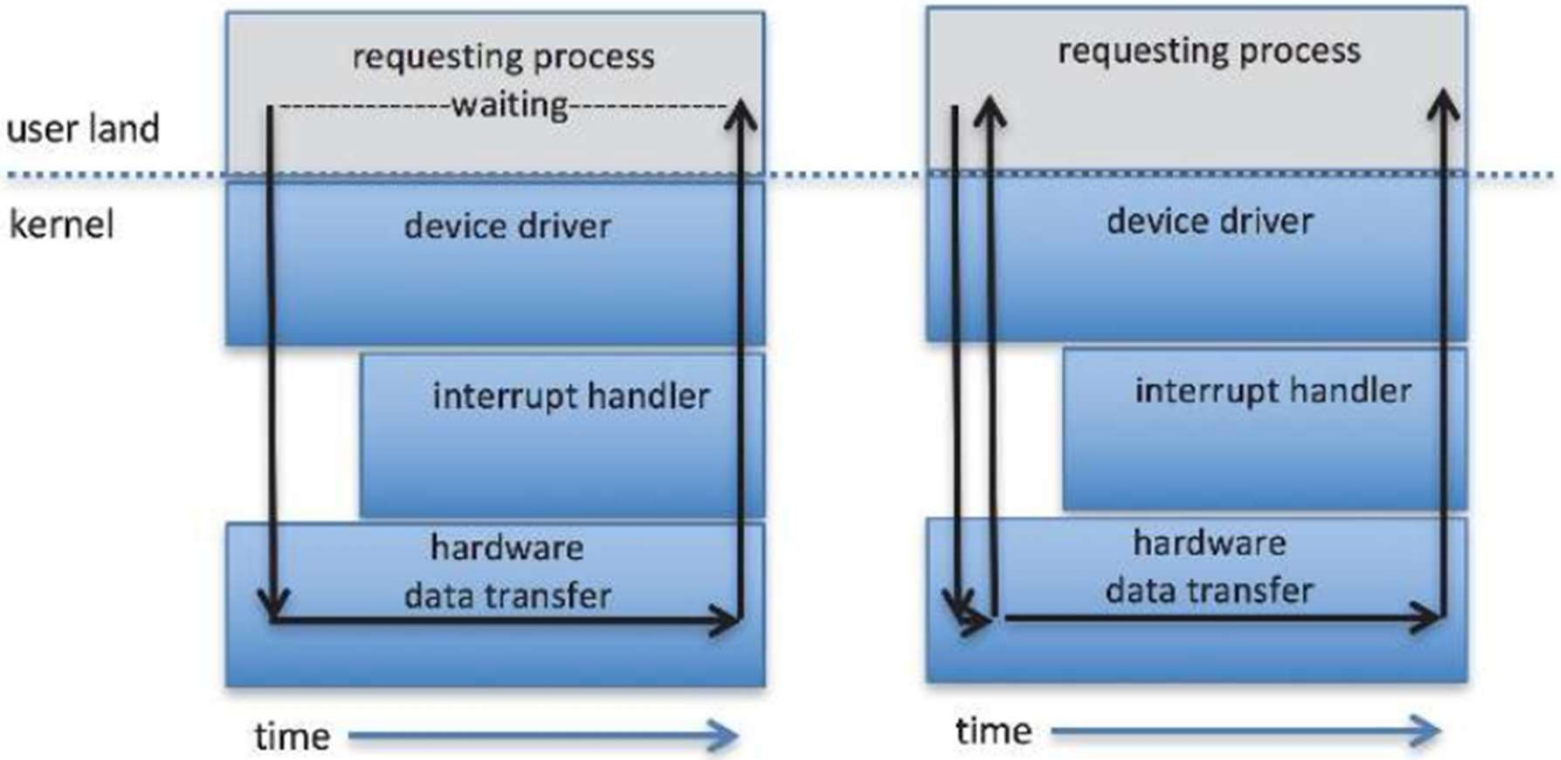
Non-blocking I/O

- **How it works:** In **non-blocking I/O**, the system call returns immediately, even if the I/O operation has not been completed
  - The system call may return a status code (e.g., EWOULDBLOCK or EAGAIN) to indicate that the operation is still in progress
- This allows the application to **continue to execute other tasks** while the I/O is happening in the background
  - The application must then periodically check the status of the I/O operation to see if it has finished. This is often done using a polling loop
- **Example:** A non-blocking read() call will return any data that is currently available, or an error if no data is ready
  - The program can then go on to do other work and come back later to read more data

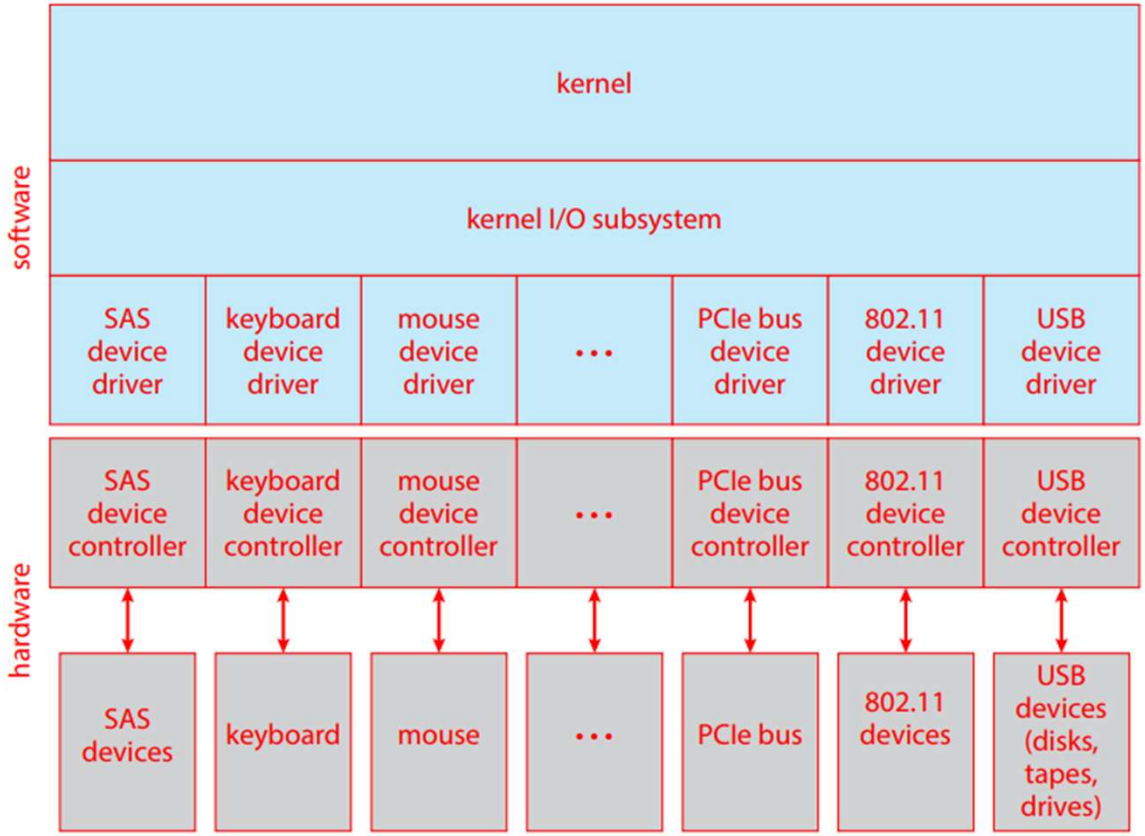
Asynchronous I/O

- **How it works: Asynchronous I/O** is a more advanced form of non-blocking I/O
  - The application issues an I/O request and then continues with its work
  - When the I/O operation is completed, the operating system notifies the process
  - This is typically done with a **signal, a callback function, or a message**, and the process does not have to constantly check the status of the operation
- Asynchronous I/O is very efficient for applications that need to handle a large number of concurrent I/O operations (e.g., web servers)
  - It allows a single thread to manage multiple connections without blocking, maximizing resource utilization
- **Example:** A web server can request data from a database and immediately go on to handle another user's request
  - When the database returns the data, the OS sends a signal to the web server, which then processes the database response

Synchronous and Asynchronous I/O



Kernel I/O Structure



Kernel I/O Subsystem

- **Services:** The kernel's I/O subsystem provides numerous services to manage and coordinate I/O:
  - **I/O Scheduling:** Orders I/O requests to optimize device performance
  - **Buffering:** Stores data during transfer to handle speed mismatches and different data-transfer sizes
  - **Caching:** Uses a fast memory region (cache) to hold copies of frequently used data for quicker access
  - **Spooling:** Buffers output for a device that cannot accept interleaved data streams (like printers)
  - **Device Reservation:** Provides explicit facilities for coordinating exclusive access to devices.
  - **Error Handling:** Manages and recovers from device failures
- **Protection:** All I/O instructions are **privileged instructions** and can only be executed in kernel mode
  - A user program must use a system call to request I/O, which the OS then validates and performs.
- **Kernel Data Structures:** The kernel keeps track of the state of I/O components using internal data structures, such as a device-status table and per-process open-file tables

Transforming I/O Requests to Hardware Operations

- **Life Cycle of a Read Request:** The journey of a blocking read request involves a detailed, multi-step process
  1. A process issues a `read()` system call on a file.
  2. The kernel's I/O subsystem checks parameters and the buffer cache.
  3. If a physical I/O is needed, the process is moved to a wait queue.
  4. The I/O subsystem sends a request to the device driver.
  5. The device driver issues commands to the device controller.
  6. The DMA controller transfers data to memory and raises an interrupt.
  7. The interrupt handler signals the device driver that the transfer is complete.
  8. The device driver signals the kernel, which moves the process back to the ready queue.
  9. The process resumes execution when scheduled by the CPU.
- **Device Naming:** The OS maps logical file names to physical device controllers using lookup tables
  - This allows new devices to be added without recompiling the kernel

STREAMS and Performance

- **STREAMS:** A UNIX mechanism that enables an application to dynamically assemble pipelines of driver code
  - It's a full-duplex connection between a device driver and a user process, consisting of a stream head, a driver end, and optional modules in between
  - This architecture simplifies the development of device drivers and network protocols
- **I/O Performance:** I/O is a major factor in system performance. Efficiency can be improved by:
  - Reducing context switches and data copying
  - Using large transfers and smart controllers to reduce interrupt frequency
  - Employing DMA to offload work from the CPU
  - Moving processing into hardware
  - Balancing CPU, memory, and I/O performance

I/O and Display Interfaces